Sales Analytics Platform

System Architecture Specification — v2 (static dataset, no sync)

This is the build specification for the current phase: the client has provided a fixed 15-day dataset for evaluation. There is no daily sync or ETL pipeline in this phase — the dataset is loaded once, and the system is built and demoed against it. The sync/ETL service will be added as a separate phase after deployment, once the client confirms the system live. This document is written to be handed directly to a coding agent: it includes the schema DDL, the API contract, the project layout, and a build order.

Phase	Evaluation build against a static 15-day dataset — no sync/ETL yet
Deployment model	Compute on a privately owned GPU machine, data + backups on AWS
Primary users	2-3 concurrent users, single-tenant
Deferred to next phase	Sync service, ETL pipeline, “last synced” freshness indicator, scheduled ingestion monitoring

Contents

1. Architecture principles
2. Component architecture
3. Data architecture & schema (DDL)
4. Data flows
5. API contract
6. Deployment topology & networking
7. Security architecture
8. Reliability & failure handling
9. Scalability & performance
10. Observability
11. Environments & release process
12. Technology stack
13. Project structure
14. Environment variables
15. Build order (for the coding agent)
16. Risk register
17. Open decisions

1. Architecture principles

- **Volatile facts live in the database, not in the model.** The LLM never memorizes figures — it queries structured data and reasons over the result.
- **No LLM-generated code executes unsandboxed.** Any code the orchestrator generates to answer a novel analytical question runs isolated, resource-capped, with no network access.
- **Client data never leaves infrastructure the client has agreed to.** Compute stays on the private GPU machine; only the warehouse and backups sit on AWS.
- **Every external boundary fails closed.** An unreachable LLM or database outage produces a visible error, never a silently wrong or empty answer.
- **Built to have sync bolted on, not rebuilt for it.** Even though ingestion is a one-time load right now, the schema, the loader script, and the data-access layer are written so a scheduled sync service can be dropped in later without changing how the API or frontend read data.

Note on scope: everything in this document describes the full target architecture for this phase, with the sync/ETL component explicitly deferred. Section 4.3 and Section 15 call out exactly what a future sync phase will need to slot into.

2. Component architecture

2.1 React frontend

Chat interface and Generate Report form (see the Frontend Requirements deck). Talks only to the FastAPI layer over HTTPS.

2.2 FastAPI service layer

- Request validation (Pydantic) — malformed requests rejected before touching the orchestrator or database.
- Routing: chat → LangGraph orchestrator; report requests → Report Engine; metadata (routes/SKUs/customers) → direct scoped read queries, bypassing the LLM.
- Response shaping into the API contract in Section 5, including the report-eligibility flag for chat-to-report handoff.
- Request/response logging with correlation IDs (Section 10).

2.3 LangGraph orchestrator

Owns the reasoning flow for a chat query: interpret → plan → execute → verify → respond, as an explicit graph rather than one opaque prompt, so each stage is independently testable and loggable.

2.4 Report Engine

Deterministic service, separate from the conversational orchestrator. Builds reports via fixed, parameterized queries — not free-form LLM generation — so figures are reproducible. The free-text notes field is the one place the LLM adjusts presentation of an already-correct dataset.

2.5 LLM inference service (Qwen via vLLM)

Self-hosted on the private GPU machine, reachable only from the FastAPI/LangGraph process on an internal interface — never exposed publicly.

2.6 PostgreSQL data warehouse

Star-schema warehouse on AWS (managed instance). Same schema whether the data behind it is a one-time 15-day load or, later, a daily-synced multi-year history — see Section 3.

2.7 Data load (this phase: one-time, manual trigger)

Replaces the sync service for now. A standalone script (not a scheduled job) that reads the client's 15-day Excel/Tally export, resolves it through the same alias-mapping tables the future sync will use, and loads it into the warehouse once. It is intentionally built as the same transformation logic a scheduled sync would call — so that adding sync later is “wrap this in a scheduler,” not “rewrite the ingestion logic.” See Section 4.3 and Section 15.

3. Data architecture & schema (DDL)

Star schema: one fact table, four dimension tables, plus a customer-alias resolution table. This is unchanged by whether ingestion is a one-time load or a daily sync — the schema is the stable contract; only what writes to it changes later.

```
CREATE TABLE dim_date (
    date_id       DATE PRIMARY KEY,
    day_of_week   SMALLINT NOT NULL,        -- 1=Mon .. 7=Sun
    day_name      VARCHAR(9) NOT NULL,
    week_number   SMALLINT NOT NULL,
    month         SMALLINT NOT NULL,
    year          SMALLINT NOT NULL
);

CREATE TABLE dim_route (
    route_id      SERIAL PRIMARY KEY,
    route_name    VARCHAR(100) NOT NULL UNIQUE,
    is_active     BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE dim_sku (
    sku_id        SERIAL PRIMARY KEY,
    sku_name      VARCHAR(150) NOT NULL,
    branch        VARCHAR(20) NOT NULL CHECK (branch IN ('coffee','tea','others')),
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE dim_customer (
    customer_id     SERIAL PRIMARY KEY,
    canonical_name  VARCHAR(200) NOT NULL,
    is_active       BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE customer_alias (
    alias_id      SERIAL PRIMARY KEY,
    customer_id   INT NOT NULL REFERENCES dim_customer(customer_id),
    source_name   VARCHAR(200) NOT NULL,     -- name as it appears in the raw export
    UNIQUE(source_name)
);

CREATE TABLE customer_route_map (
    customer_id   INT NOT NULL REFERENCES dim_customer(customer_id),
    route_id      INT NOT NULL REFERENCES dim_route(route_id),
    PRIMARY KEY (customer_id, route_id)
);

CREATE TABLE fact_sales (
    fact_id       BIGSERIAL PRIMARY KEY,
    date_id       DATE NOT NULL REFERENCES dim_date(date_id),
    route_id      INT NOT NULL REFERENCES dim_route(route_id),
    sku_id        INT NOT NULL REFERENCES dim_sku(sku_id),
    customer_id   INT NOT NULL REFERENCES dim_customer(customer_id),
    quantity      NUMERIC(12,2) NOT NULL,
    value         NUMERIC(14,2) NOT NULL,
    loaded_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    load_batch_id UUID NOT NULL              -- ties every row to the load run that inserted
 it
);

CREATE INDEX idx_fact_sales_date     ON fact_sales(date_id);
CREATE INDEX idx_fact_sales_route    ON fact_sales(route_id);
CREATE INDEX idx_fact_sales_sku      ON fact_sales(sku_id);
CREATE INDEX idx_fact_sales_customer ON fact_sales(customer_id);
```

load_batch_id on every fact row is what makes the one-time load safely re-runnable and auditable: if the 15-day file is reloaded during development, the previous batch can be identified and deleted cleanly instead of guessing

which rows came from which run. This same column is what a future sync service will populate per run.

4. Data flows

4.1 Chat query flow

1. Frontend sends the message to **POST /chat**.
2. FastAPI validates and forwards to the LangGraph orchestrator with conversation context.
3. Orchestrator interprets intent and either maps it to a parameterized SQL query, or — for a question the fixed query set can't answer — generates scoped analysis code that runs in the sandbox (Section 7.3) against a read-only snapshot.
4. Result is passed back through the LLM to produce a natural-language answer, optionally with a table/chart payload.
5. If the filters map cleanly to the report field set, the response is tagged report-eligible.
6. FastAPI returns the composed response; frontend renders it and, if flagged, shows “Generate this as a report.”

4.2 Report generation flow

1. Frontend sends selected filters to **POST /reports**.
2. FastAPI validates required fields (date range, output format) and forwards to the Report Engine.
3. Report Engine resolves defaults (untouched fields → “all”) and runs fixed, parameterized queries — no free-form generation for the figures.
4. If notes were provided, the LLM is consulted once to adjust presentation without altering the numbers.
5. If Generate Chart is on, the chart is rendered server-side.
6. Output is assembled (Excel/PDF) and returned as a download; empty result sets return an explicit “no data” response.

4.3 One-time data load (this phase — sync deferred)

1. Developer/operator places the client's 15-day Excel/Tally export in a known input location and runs scripts/load_initial_data.py manually (see Section 15).
2. Script validates rows against expected schema/types; malformed rows are written to a quarantine file with a reason, not silently dropped or silently inserted.
3. Customer and SKU names are resolved through customer_alias / dim_sku; unresolvable names stop the load for that row and are reported, not guessed at.
4. Valid rows are inserted into fact_sales inside a single transaction tagged with a new load_batch_id, so a failure partway through rolls back cleanly.
5. Script prints a summary: rows loaded, rows quarantined, date range covered, batch id — this is the manual equivalent of the sync-status record a scheduled service would write automatically later.

What changes when sync is added later: this same validation/resolution/transactional-insert logic gets called by a scheduler instead of by hand, and its summary gets written to a sync_log table and wired to alerting instead of printed to a terminal. Building the loader as a clean, callable function now (not an inline script) means that swap is small.

5. API contract

Concrete request/response shapes for the coding agent to implement against. Field names here should be treated as the source of truth for both frontend and backend.

POST /chat

```
// Request
{
  "message": "which route did the most coffee volume last week",
  "conversation_id": "uuid-or-null"
}

// Response
{
  "conversation_id": "uuid",
  "answer_text": "Route 3 led coffee volume last week with 1,240 units.",
  "table": null,               // or { "columns": [...], "rows": [...] }
  "chart": null,                // or { "type": "bar", "data": {...} }
  "report_eligible": true,
  "report_prefill": {
    "date_from": "10/08/2026",
    "date_to": "16/08/2026",
    "days": ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"],
    "route_ids": [3],
    "branches": ["coffee"],
    "sku_ids": null,
    "customer_ids": null
  }
}
```

POST /reports

```
// Request
{
  "date_from": "01/08/2026",
  "date_to": "15/08/2026",
  "days": ["Mon","Fri"],              // omit or empty = all days
  "route_ids": [1,3],                 // omit or empty = all routes
  "branches": ["coffee","tea"],       // "others" and/or specific sku_ids
  "sku_ids": null,
  "customer_ids": null,               // omit or empty = all customers in selected routes
  "generate_chart": true,
  "chart_type": "bar",                // placeholder set, to be finalized
  "output_format": "excel",           // "excel" | "pdf"
  "notes": "Highlight top 5 customers by value"
}

// Response: binary file stream
// Content-Disposition: attachment; filename="report_2026-08-01_2026-08-15.xlsx"
// If zero rows match filters, respond 200 with:
{ "status": "no_data", "message": "No records found for the selected filters." }
```

GET /meta/routes, /meta/skus, /meta/customers

```
GET /meta/routes
-> [{ "route_id": 1, "route_name": "Route 1" }, ...]

GET /meta/skus?branch=coffee
-> [{ "sku_id": 12, "sku_name": "Filter Coffee 250g", "branch": "coffee" }, ...]

GET /meta/customers?route_ids=1,3
-> [{ "customer_id": 88, "canonical_name": "Sunrise Traders" }, ...]

GET /health
-> { "status": "ok", "db": "ok", "llm": "ok" }
```

Meta endpoints are plain read queries against the dimension tables — they must not go through the LLM or the orchestrator. This keeps the dropdowns in the frontend fast and independent of LLM availability.

6. Deployment topology & networking

- **Private GPU PC:** FastAPI, LangGraph orchestrator, Report Engine, and vLLM/Qwen, each a separate Docker container behind an nginx/Caddy reverse proxy terminating TLS.
- **AWS:** PostgreSQL (managed instance) and automated backups only — no application logic on AWS in this phase.
- **Connectivity:** private machine → PostgreSQL over an encrypted connection, restricted by IP allowlist/security group to the private machine's address — not open to the public internet.
- **Inbound access:** only the reverse proxy accepts inbound connections, over HTTPS. The LLM inference port and internal services are never exposed externally.

7. Security architecture

7.1 Secrets management

- DB credentials, API keys, and tokens live in environment variables (Section 14) or a secrets manager — never committed to source control.
- Separate credentials per environment (dev/staging/prod).

7.2 Least-privilege database access

- The API's database role has SELECT on warehouse tables only — not a superuser/owner role.
- The data-load script uses a separate role with INSERT/UPDATE scoped to the tables it needs, kept distinct from the query-serving API role.

7.3 Sandboxing for generated code

The highest-risk component regardless of ingestion phase: the orchestrator can generate and execute code for novel analytical questions. That execution is treated as untrusted, always.

- Runs in an ephemeral, resource-capped container (CPU/memory/time limits), destroyed after each execution.
- No network access from inside the sandbox.
- Filesystem access limited to a read-only snapshot of the data needed — never a live production DB connection from inside the sandbox.
- Hard timeout and output-size cap; failures return a clean error, not a hang.

7.4 Transport & auth

- All frontend ↔ FastAPI traffic over HTTPS (TLS 1.2+).
- No user login in this phase (per current scope) — access control is enforced at the network boundary (VPN/IP allowlist), an explicit and temporary trade-off, not a permanent stance.
- Audit logging (Section 10) still applies without per-user login, so “what query produced this report” stays answerable.

8. Reliability & failure handling

8.1 Data load validation (this phase)

- The load script is transactional (Section 4.3) — a partial failure rolls back rather than leaving the warehouse half-updated.
- Row-level validation errors are quarantined with a reason and reported in the load summary, never silently skipped.
- Because this is a manual, one-time load rather than a scheduled job, there's no automated alerting needed yet — the operator running the script sees the summary directly. This gap is exactly what the future sync phase closes (Section 15).

8.2 Service-level resilience

- FastAPI enforces request timeouts on calls to the orchestrator and the LLM inference service.
- Retries with backoff for transient failures (e.g. a momentary DB connection blip); LLM generation failures surface as explicit errors instead of being retried blindly.
- Database connection pooling sized for 2-3 concurrent users, with headroom.

8.3 Backup & recovery

- Automated daily snapshots plus WAL archiving on the managed PostgreSQL instance for point-in-time recovery.
- Backup restore is tested periodically, not just configured and assumed to work.
- The original 15-day source file is retained outside the database, so the one-time load can be replayed from source if needed.

9. Scalability & performance

- Stateless FastAPI/orchestrator processes (chat state client-side or in a lightweight store) so the compute layer can scale to multiple instances later without a rearchitecture.
- Report generation runs as an async/background job with a job-status endpoint rather than holding the HTTP request open, so larger exports don't block or time out.
- Indexes on fact_sales(date_id, route_id, sku_id, customer_id) keep aggregation fast even though the current dataset is small — this pays off the moment real, multi-month data lands.

10. Observability

- **Structured logging** across FastAPI, the orchestrator, and the Report Engine, with a correlation/request ID tying a chat message or report request to every downstream log line.
- **Metrics:** request latency and error rate per endpoint, LLM inference latency, sandbox execution time and failure rate.
- **Alerting:** LLM inference service down, database connection exhaustion, sandbox timeout rate spiking.
- **Audit trail:** every report and chat query logged with its resolved filters/query and timestamp, traceable back to exactly what was asked and what data it drew from.
- **Deferred:** sync-failure alerting and a “last synced” freshness indicator — both come back in the sync phase (Section 15). Until then, data freshness is simply “as of the 15-day file that was loaded,” which is fine for an evaluation build but should not be silently carried into production framing.

11. Environments & release process

- **Local/dev:** docker-compose bringing up FastAPI, orchestrator, a local Postgres seeded via the same loader script against a small synthetic sample — never real client data in dev.
- **Staging:** mirrors production topology with a staging database, used to validate schema changes before they touch the client's data.
- **Production:** as described in Section 6, loaded once via Section 4.3 for this phase.
- **Migrations:** schema changes go through Alembic with reviewed, versioned migration files — no manual ALTER TABLE against production.
- **Version control:** all services in git, orchestrator prompt/graph logic versioned alongside the code that calls it.

12. Technology stack

Layer	Choice / rationale
Frontend	React, static build served via the reverse proxy
API layer	FastAPI (Python), Pydantic, Uvicorn/Gunicorn
Orchestration	LangGraph for explicit, inspectable multi-step reasoning
LLM inference	Qwen served via vLLM
Sandbox	Ephemeral, resource-capped containers, no network, for generated-code execution
Database	PostgreSQL (managed instance on AWS, e.g. RDS), star-schema warehouse
Backups	Automated snapshots + WAL archiving
Data load	Standalone Python script (this phase); designed to be wrapped by a scheduler later
Report output	openpyxl for Excel, ReportLab/WeasyPrint for PDF
Reverse proxy / TLS	nginx or Caddy on the private GPU machine
Containerization	Docker for every service; docker-compose for local/staging
Observability	Structured JSON logs to a log aggregator; Prometheus-compatible metrics

13. Project structure

Reference layout for the coding agent — keep the loader in scripts/, not inside the API app, so it stays a clean, separately callable unit for the future sync swap.

```
/backend
  /app
    /api              # FastAPI routers: chat.py, reports.py, meta.py, health.py
    /core             # config.py, security.py, logging.py
    /orchestration    # LangGraph graph + node definitions
    /report_engine    # report_builder.py, excel_export.py, pdf_export.py, chart_render.py
    /db               # models.py, session.py
    /migrations       # Alembic migration files
    /sandbox          # sandbox_runner.py, container config
    /schemas          # Pydantic request/response models
    main.py
  /scripts
    load_initial_data.py   # one-time loader (Section 4.3) -- callable, not a throwaway scri
pt
  Dockerfile
  docker-compose.yml
  requirements.txt / pyproject.toml
  .env.example

/frontend
  /src
    /components
    /pages            # Chat.tsx, GenerateReport.tsx
    /api               # client.ts, matches Section 5 contract
  package.json
  Dockerfile

/infra
  nginx.conf
  docker-compose.prod.yml
```

14. Environment variables

Variable	Purpose
DATABASE_URL	postgresql connection string, API role
DATABASE_URL_LOADER	postgresql connection string, load-script role (separate privileges)
LLM_ENDPOINT	internal URL of the vLLM/Qwen inference service
SANDBOX_IMAGE	container image used for generated-code execution
SANDBOX_TIMEOUT_SECONDS	hard timeout for sandboxed execution
CORS_ALLOWED_ORIGINS	frontend origin(s) allowed to call the API
LOG_LEVEL	structured logging verbosity per environment
ENVIRONMENT	dev / staging / production

15. Build order (for the coding agent)

Sequenced so each step is independently testable before the next depends on it.

1. Scaffold the repo per Section 13; set up docker-compose for local Postgres.
2. Apply the schema DDL (Section 3) via an initial Alembic migration.
3. Write scripts/load_initial_data.py and load the 15-day dataset into local Postgres; verify row counts and spot-check a few aggregates by hand.
4. Build FastAPI skeleton with /health and the three /meta endpoints (Section 5) reading directly from the warehouse — no LLM involved yet. This alone lets the frontend's dropdowns be built and tested.
5. Build the Report Engine's fixed-query path and Excel/PDF export, tested directly against the loaded dataset, independent of the LLM.
6. Stand up the vLLM/Qwen inference service and confirm it's reachable only internally.
7. Build the LangGraph orchestrator's chat flow (interpret → plan → execute → verify → respond) against the fixed-query path first, before adding sandboxed code generation.
8. Add the sandbox for novel/generated-code queries, with the isolation constraints in Section 7.3 enforced from the first version — not added later.
9. Wire the chat-to-report handoff (report_eligible / report_prefill in the /chat response).
10. Build the React frontend against the API contract in Section 5.
11. Add structured logging and the metrics/alerting in Section 10.
12. Deploy behind the reverse proxy per Section 6; confirm only FastAPI's port is externally reachable.
13. (Next phase, not now) Retrofit the loader from Section 4.3 into a scheduled sync service once the client confirms live deployment — the loader's validation/resolution/transactional-insert logic should not need to change, only what calls it.

16. Risk register

Risk	Severity	Mitigation
Unsandboxed LLM-generated code execution	High	Ephemeral, network-isolated, resource-capped sandbox per Section 7.3 — no exceptions.
One-time load treated as “done forever” and sync retrofit forgotten post-deployment	Medium	Loader built as a callable, reusable function (Section 4.3, 15) with the deferral explicitly tracked in Section 17, not left implicit.
Report figures diverging from chat-query figures due to two code paths	Medium	Report Engine and the orchestrator's structured-query path share the same query-building functions for common filters.
No user auth means network boundary is the only access control	Medium	Explicit, documented trade-off; revisit before onboarding additional clients or wider exposure.
Private-machine downtime with no cloud failover	Medium	Document and test a recovery runbook; consider a standby cloud GPU path if uptime needs tighten.
Alias-mapping gaps as new customer name variants appear once real ingestion starts	Low (for now)	Loader stops and reports unresolvable names rather than guessing; becomes more relevant once sync is added.

17. Open decisions

- Whether the 7 routes are truly static or should become DB-driven like SKUs.
- Whether chat history should persist across page reloads.
- Static IP vs dynamic-DNS/tunnel strategy for the private machine's public reachability.
- **Sync/ETL retrofit:** confirmed deferred until after deployment and client sign-off, per this conversation — tracked here so it isn't lost. When it's time, extend Section 4.3's loader into a scheduled service, add the deferred observability items from Section 10, and reintroduce the sync-failure risk from the earlier full architecture pass.
- When (not if) user authentication gets added, and whether it's single-role or multi-role.
- Whether additional clients will share this architecture or each gets an isolated deployment — assumed isolated per-client for now.

End of document.